

CSC1029 OO Programming

Lecture 6 – Objects, Encapsulation & Lifecycle





Some tips for an easier life ...

- ✓ Remember to read your emails - at least twice a day!
- ✓ For every lecture - you should be studying for at least 1 additional hour
- ✓ Practical exercises will require you to work outside the lab time.

Recap – What do we know about OOP? ⁽¹⁾

- ✓ An executing *OOP* program is a collection of interacting objects
- ✓ An *object* is a software representation of a real-world entity or concept
- ✓ Objects are defined using the *class* structure
- ✓ A class is a 'blueprint' for creating objects of a specific type
- ✓ An object is an *instance* of a class
- ✓ The process of object creation is called *instantiation*

Recap – What do we know about OOP? ⁽²⁾

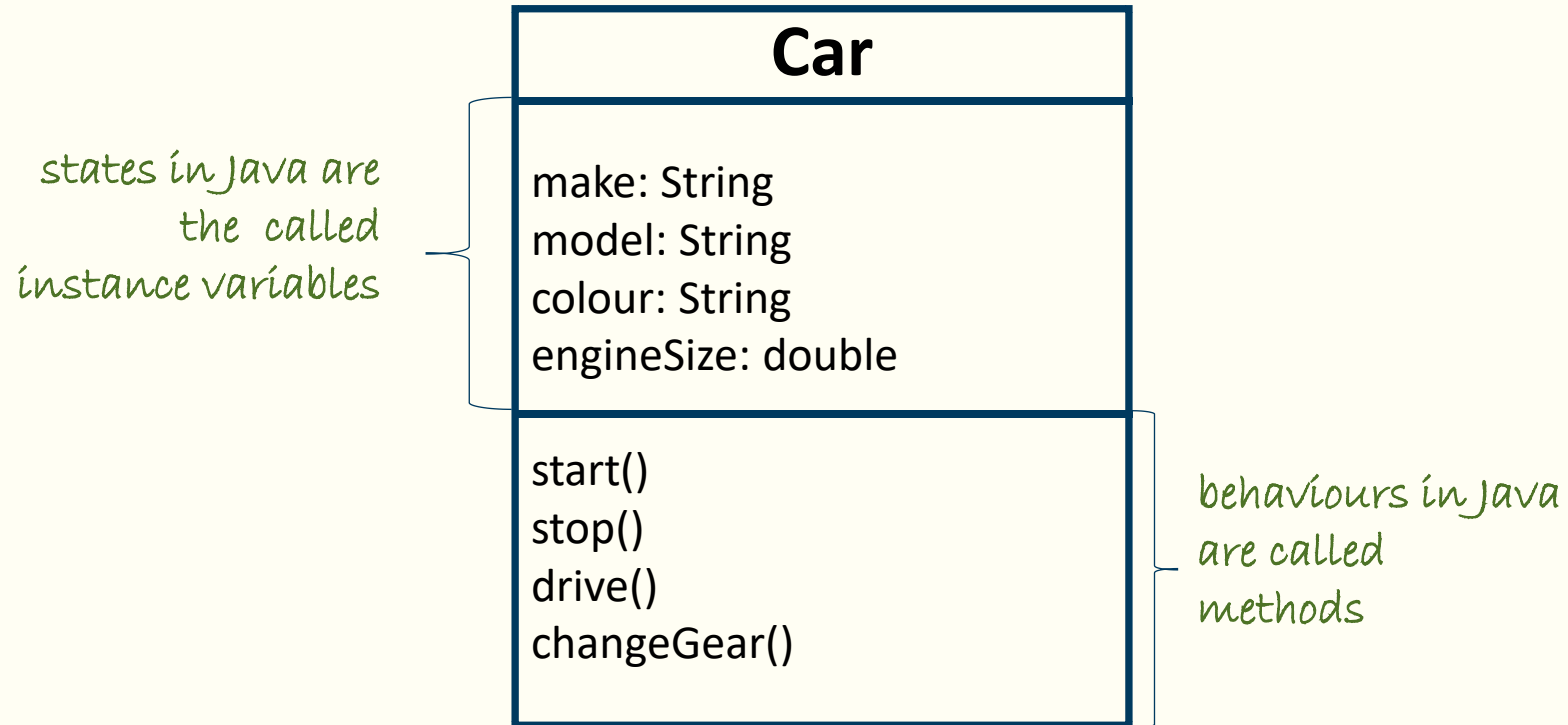
- ✓ Objects are defined in terms of *state* and *behaviour*
- ✓ *Instance variables* may be used to represent *state*
- ✓ Instance variables typically have different values in different objects
- ✓ *Instance methods* may be used to represent *behaviour*
- ✓ Instance methods are *not* defined using the *static* keyword
- ✓ *Static* methods are not associated with object instances



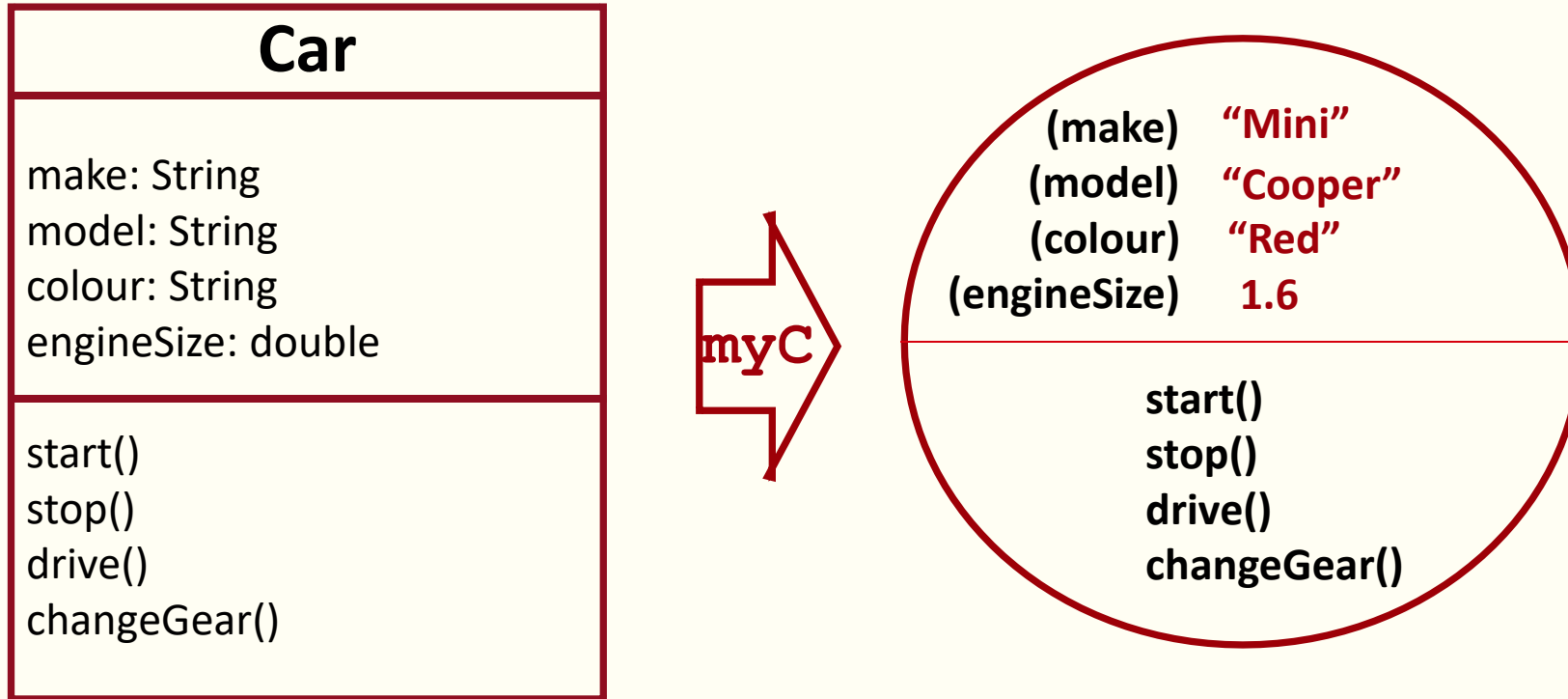
What's to come in today's lecture?

- object lifecycle - construction
- access modifiers
- getters and setters
- testing objects
- object lifecycle - destruction
- data scope

The Car Class



Using a Class – Creating an Object



```
Car myC = new Car() ; // an instance of Car
myC.make = "Mini" ; // set make
myC.model = "Cooper" ; // set model
myC.colour = "Red" ; // set colour
myC.engineSize = 1.6 ; // set engineSize
```

Object Construction₍₁₎

- Constructors are used in building objects.
- They are special methods used to initialise object data (instance variables).
- Syntax is different from other methods – how & why?





Object Construction₍₂₎

- A constructor initializes an object when it is created.
 - `Car myC = new Car();`
- A constructor is a special type of method and will share the class name. No return type?

```
public Car() {  
    //statements  
}
```

Object Construction⁽³⁾

```
public class Car {  
    public String make;  
    public String model;  
    public double engineSize;  
  
    public Car() {  
        // a default constructor  
    }  
    public void drive() {  
        System.out.println("I am driving");  
    }  
}
```

There is no visible constructor but Java “places” it there when compiling

Object Construction⁽⁴⁾

```
public class Car {  
    public String make;  
    public String model;  
    public double engineSize;
```

```
    public Car() {  
        make = "";  
        model = "";  
        engineSize = 0.0;  
    }
```

← This is an example of what the
Java constructor would look like

```
    public void drive() {  
        System.out.println("I am driving");  
    }  
}
```



Object Construction⁽⁵⁾

- We can define our own constructors
- The *parameter list* describes the information the constructor needs to work

```
public Car(String newMake, String newModel,  
           double newEngSize) {  
    make = newMake;  
    model = newModel;  
    engineSize = newEngSize;  
}
```

Object Construction⁽⁶⁾

```
public class Car {  
    public String make;  
    public String model;  
    public double engineSize;
```

```
    public Car(String newMake, String newModel, double newEngSize)  
    {  
        make = newMake;  
        model = newModel;  
        engineSize = newEngSize;  
    }
```

```
    public void drive() {  
        System.out.println("I am driving");  
    }  
}
```

This is an example of what our constructor would look like

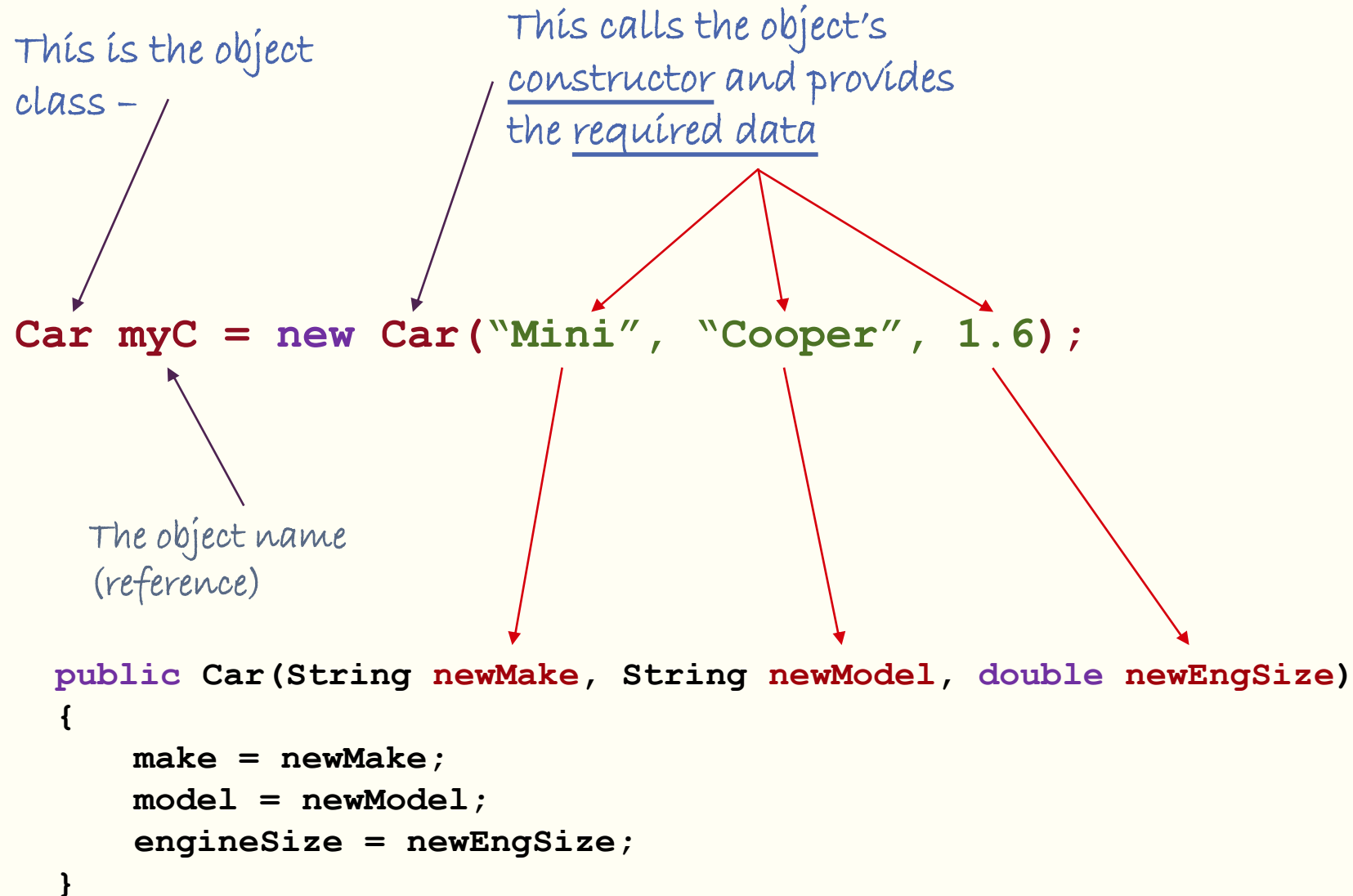


Changes to the Test Program

```
public class CarTester {  
    public static void main (String[] args) {  
        //create an object of type car  
        Car myC = new Car("Mini", "Cooper", 1.6);  
  
        //print the make of the car  
        System.out.println(myC.make);  
  
        //test the drive method  
        myC.drive();  
    }  
}
```



Object Construction



Access Modifiers⁽¹⁾

Car

make: String
model: String
colour: String
engineSize: double

start()
stop()
drive()
changeGear()

```
Car myC = new Car() ;  
myC.make = "Mini" ;  
myC.model = "Cooper" ;  
myC.colour = "Red" ;  
myC.engineSize = 1.6 ;
```

In this example, we can **directly** assign/modify the values of instance variables via the object reference.

Why is this a bad idea?

What's the alternative?

Access Modifiers⁽²⁾

```
public class Car {  
    public String make;  
    public String model;  
    public double engineSize;  
  
    public Car(String newMake,  
                String newModel,  
                double newEngSize) {  
        make = newMake;  
        model = newModel;  
        engineSize = newEngSize;  
    }  
  
    public void drive() {  
        System.out.println("I am driving");  
    }  
}
```

`myC.engineSize = 1.6 ;`

public is an access modifier

Use the **private** access modifier to prevent direct external access.

private double engineSize ;

`myC.engineSize = 1.6 ;`

How do we access private data?



Getters & Setters⁽¹⁾

```
public class Car {  
    private String make;  
    private String model;  
    private double engineSize;  
  
    public Car( ... ) {  
    }  
  
    public String getMake() {  
        return make;  
    }  
  
    public String getModel() {  
        return model;  
    }  
  
    public double getEngineSize() {  
        return engineSize;  
    }  
}
```

```
String mod = myCar.getModel();  
String mk = myCar.getMake();  
double sz = myCar.getEngineSize();
```

Should you provide getter
methods for
all instance data?

Getters & Setters ⁽²⁾

```
public class Car {  
    private String make;  
    private String model;  
    private double engineSize;  
  
    public Car( ... ) {  
    }  
    public void setEngineSize(double sz)  
    {  
        if ( sz >= MIN_SIZE ) {  
            engineSize = sz;  
        }  
    }  
}
```

```
myCar.setEngineSize(1.8);
```

Should you provide setter methods for all instance data?

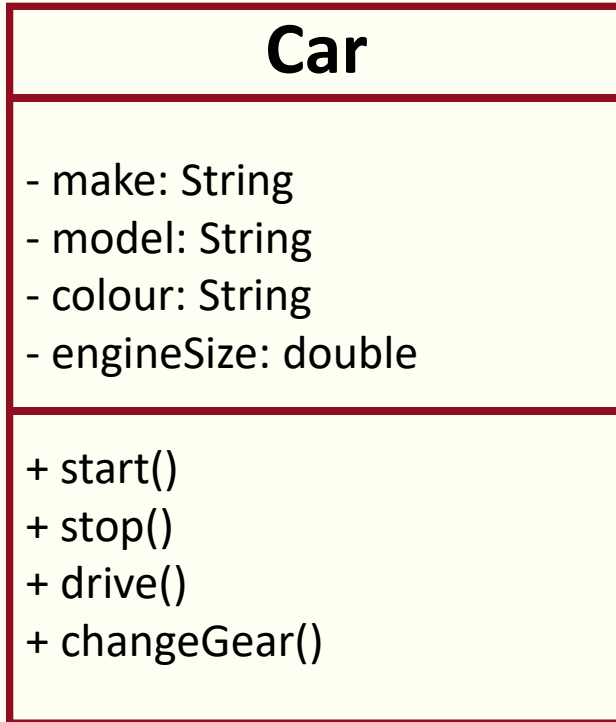
Why is it important to *validate* data used with a setter?

Encapsulation

Encapsulation is a fundamental concept of OOP –

- describes the idea of bundling **data** and **methods** that work on that data within one unit – called a class.
- often involves **hiding** the data (or state) of an object from external manipulation
- will involve the use of **getters** and **setters** to strictly control how information is accessed
- **all data should be hidden** from external view (marked private) – unless you have a good justifiable reason for anything else.

Access Modifiers & UML



UML = Universal Modelling Language
(We'll see quite a bit more of UML later.)

Used to represent classes in OOP.

Access modifiers –

public (+) and private (-)

Applied to *data* and *methods*.

Testing Objects

```
public class Car {  
    private String make;  
    private String model;  
    private double engineSize;  
  
    public Car( ... ) {  
    }  
    public void setEngineSize(double sz)  
    {  
        if ( sz >= MIN_SIZE ) {  
            engineSize = sz;  
        }  
    }  
}
```

```
public class CarTester{  
    public static void main(String args[])  
    {  
        Car myC = new Car( ... );  
        myC.setEngineSize(1.8);  
  
        ...  
    }  
}
```

Always good to create a separate class/program for testing purposes.

Initial testing is normally performed by the code developer - Unit Testing

Example



Car.java

CarTester.java



Object Lifecycle - Destruction

Some OOP languages (e.g. C++) provide an explicit method (called a destructor) - automatically called when an object is no longer referenced.

In Java - objects that are no longer referenced are 'garbage collected' by the system (Java Runtime Environment - JRE).

A 'finalize' method may be added to a class - which will be called on 'garbage collection'

This process is outside programmer control.



Exercise⁽¹⁾

A Calculator object class

Calculator
- x : int - y : int
+ Constructor... + Getters... + Setters... + add() + subtract() + multiply() + divide()

- The Calculator class
- Defines 2 instance variables
- Defines 4 behaviour methods for adding, subtracting, multiplying and dividing the instance variables
- Each method should(using x and y) calculate and display an answer, displayed in a suitable message

Exercise ⁽²⁾

A Calculator object class

Calculator
- x : int - y : int
+ Constructor... + Getters... + Setters... + add() + subtract() + multiply() + divide()

- The CalculatorTester class
- Defines a *main* method
- Using a *Scanner*, asks the user for *2 integers – first & second*
- Uses these integers to create an instance of Calculator
- Test each method in turn (*add, subtract, multiply, divide*)



What's Next?

- Class data & Class Methods
- More on object References
- Java Arrays
- Exception Handling
- A Menu Case-Study